Fundação Getulio Vargas
Escola de Matemática Aplicada
Curso de Graduação em Ciência de Dados e Inteligência Artificial

Sistema de Apoio a Atendimento de Chamados via GraphRAG com Grafo de Conhecimento em Neo4j

por Yonathan Rabinovici Gherman

Rio de Janeiro - Brasil
2025

Fundação Getulio Vargas
Escola de Matemática Aplicada
Curso de Graduação em Ciência de Dados e Inteligência Artificial

Sistema de Apoio a Atendimento de Chamados via GraphRAG com Grafo de Conhecimento em Neo4j

"Declaro ser o único autor do presente projeto de monografia que refere-se ao plano de trabalho a ser executado para continuidade da monografia e ressalto que não recorri a qualquer forma de colaboração ou auxílio de terceiros para realizá-lo a não ser nos casos e para os fins autorizados pelo professor orientador."

Yonathan Rabinovici Gherman

Rio de Janeiro - Brasil
2025

Fundação Getulio Vargas
Escola de Matemática Aplicada
Curso de Graduação em Ciência de Dados e Inteligência Artificial

Sistema de Apoio a Atendimento de Chamados via GraphRAG com Grafo de Conhecimento em Neo4j

"Projeto de Monografia apresentado à Escola de Matemática Aplicada como requisito parcial para continuidade ao trabalho de monografia."

Aprovado em ____ de _____ de _____

Grau atribuído ao Projeto de Monografia: ____

Professor Orientador: Júlio César Chaves
Escola de Matemática Aplicada
Fundação Getúlio Vargas

Resumo

O suporte técnico em Tecnologia da Informação (TI) enfrenta desafios constantes relacionados à triagem eficiente de chamados e à resolução de problemas que dependem de informações contextuais frequentemente omitidas pelos usuários em suas solicitações iniciais. O avanço dos Grandes Modelos de Linguagem (LLMs) permitiu o desenvolvimento de novas abordagens para automação, dentre as quais se destaca a Geração Aumentada por Recuperação (RAG). Contudo, sistemas de RAG tradicionais baseados puramente em vetores muitas vezes falham em capturar as regras de negócio e dependências explícitas necessárias para um diagnóstico preciso. Este trabalho propõe o desenvolvimento de um sistema baseado em **GraphRAG**, que integra um Grafo de Conhecimento estruturado no banco de dados Neo4j com modelos generativos (GPT-4o). O sistema utiliza uma abordagem híbrida, combinando busca vetorial para identificação de similaridade semântica e travessia de grafos para identificar informações faltantes cruciais (como número de patrimônio, modelo de hardware ou localização) antes de sugerir uma solução. A metodologia envolveu a estruturação de uma base de conhecimento de incidentes comuns, a ingestão desses dados em uma instância Neo4j AuraDB e o desenvolvimento de um pipeline de orquestração em Python. Os resultados indicam que a abordagem aumenta significativamente a completude dos chamados abertos, reduzindo o tempo de interação humana no primeiro nível de suporte e padronizando o atendimento.

Palavras-chave: GraphRAG, Neo4j, Grafo de Conhecimento, Suporte Técnico, LLM, Inteligência Artificial.

Abstract

IT technical support faces constant challenges regarding the efficient triage of tickets and the resolution of problems that depend on contextual information often omitted by users in their initial requests. The advancement of Large Language Models (LLMs) has enabled the development of new approaches for automation, among which Retrieval-Augmented Generation (RAG) stands out. However, traditional RAG systems based purely on vectors often fail to capture the business rules and explicit dependencies necessary for an accurate diagnosis. This work proposes the development of a system based on **GraphRAG**, which integrates a Knowledge Graph structured in the Neo4j database with generative models (GPT-4o). The system uses a hybrid approach, combining vector search for semantic similarity identification and graph traversal to identify crucial missing information (such as asset number, hardware model, or location) before suggesting a solution. The methodology involved structuring a knowledge base of common incidents, ingesting this data into a Neo4j AuraDB instance, and developing an orchestration pipeline in Python. The results indicate that the approach significantly increases the completeness of opened tickets, reducing human interaction time at the first level of support and standardizing service.

Keywords: GraphRAG, Neo4j, Knowledge Graph, Technical Support, LLM, Artificial Intelligence.

Lista de Figuras

1 Esquema de modelagem proposto para o Grafo de Suporte no Neo4j, evidenciando as relações direcionadas. 14
2 Pipeline de processamento do sistema GraphRAG desenvolvido. 15

Lista de Tabelas

1 Comparativo: Banco Relacional vs. Banco de Grafos 13
2 Comparativo: Suporte Humano vs. GraphRAG 25

Lista de Abreviaturas e Siglas

API - Application Programming Interface

GraphRAG - Graph-based Retrieval-Augmented Generation

ITSM - IT Service Management

JSON - JavaScript Object Notation

LLM - Large Language Model

MVP - Minimum Viable Product

NLP - Natural Language Processing

RAG - Retrieval-Augmented Generation

SLA - Service Level Agreement

SQL - Structured Query Language

VPN - Virtual Private Network

Sumário

1 Introdução **10**
1.1 Problema de Pesquisa 10
1.2 Objetivos 10
1.2.1 Objetivo Geral 10
1.2.2 Objetivos Específicos 10
1.3 Justificativa 11

2 Referencial Teórico **12**
2.1 Grandes Modelos de Linguagem (LLMs) 12
2.1.1 Mecanismo de Atenção 12
2.2 Embeddings e Representação Vetorial 12
2.3 Retrieval-Augmented Generation (RAG) 13
2.4 Grafos de Conhecimento e GraphRAG 13
2.4.1 Comparativo de Tecnologias de Banco de Dados 13
2.5 Engenharia de Prompt 14

3 Materiais e Métodos **15**
3.1 Arquitetura do Sistema 15
3.2 Ferramentas Utilizadas 15
3.3 Estruturação e Ingestão de Dados 16
3.4 O Algoritmo de Recuperação (Retrieval) 17
3.5 Custos e Viabilidade 18

4 Resultados e Discussão **19**
4.1 Cenário 1: Hardware Crítico 19
4.1.1 Caso 1.1: Computador Inoperante 19
4.1.2 Caso 1.2: Reinicialização Inesperada 19
4.2 Cenário 2: Periféricos e Acessórios 19
4.2.1 Caso 2.1: Monitor sem Imagem 19
4.2.2 Caso 2.2: Teclado Inoperante 20
4.2.3 Caso 2.3: Mouse sem Resposta 20
4.2.4 Caso 2.4: Scanner não Reconhecido 20
4.2.5 Caso 2.5: Falha em Projetor 20
4.3 Cenário 3: Conectividade e Redes 21
4.3.1 Caso 3.1: Lentidão de Rede 21
4.3.2 Caso 3.2: Falha de Conexão Wi-Fi 21
4.3.3 Caso 3.3: Acesso VPN 21
4.4 Cenário 4: Software e Sistemas Internos 22

4.4.1 Caso 4.1: Erro de Login . . . 22
4.4.2 Caso 4.2: Erro em Software de Gestão . . . 22
4.4.3 Caso 4.3: Travamento de Software . . . 22
4.4.4 Caso 4.4: Envio de E-mails . . . 22
4.4.5 Caso 4.5: Permissão de Arquivos . . . 23
4.4.6 Caso 4.6: Falha de Backup . . . 23
4.5 Cenário 5: Manutenção de Sistema Operacional . . . 23
4.5.1 Caso 5.1: Lentidão Geral . . . 23
4.5.2 Caso 5.2: Atualização do Windows . . . 23
4.5.3 Caso 5.3: Problemas de Áudio . . . 24
4.5.4 Caso 5.4: Impressão . . . 24
4.6 Análise Geral dos Resultados . . . 24
4.7 Comparativo de Desempenho Qualitativo . . . 24
4.8 Limitações Identificadas . . . 25

5 Trabalhos Futuros e Continuidade 26
5.1 Expansão da Ontologia . . . 26
5.2 Interface de Usuário (Frontend) . . . 26
5.3 Aprendizado Contínuo (Feedback Loop) . . . 26
5.4 Continuidade via Iniciação Científica . . . 27

6 Conclusão 27

7 Referências Bibliográficas 29

1 Introdução

O avanço dos modelos de linguagem natural (NLP), impulsionado pela arquitetura Transformer e o surgimento dos Grandes Modelos de Linguagem (LLMs), tem permitido o desenvolvimento de sistemas inteligentes capazes de auxiliar na resolução de problemas complexos com uma fluidez sem precedentes. No contexto corporativo, especificamente no suporte técnico de Tecnologia da Informação (TI), a eficiência operacional é frequentemente prejudicada pela qualidade das informações fornecidas pelos usuários finais.

Muitos chamados não podem ser resolvidos de forma imediata devido à ausência de informações específicas. Um usuário pode relatar "a internet não funciona", mas para o técnico, essa informação é insuficiente sem saber se é uma conexão cabeada ou Wi-Fi, qual o local do usuário, ou se outros dispositivos estão afetados. O processo tradicional envolve uma troca de mensagens assíncrona ("ping-pong") para coletar esses dados, consumindo tempo valioso de SLA (Service Level Agreement).

Este projeto propõe o desenvolvimento de um sistema de **GraphRAG**, que utiliza um grafo de conhecimento estruturado no Neo4j para fornecer respostas automáticas a esses chamados. Diferente de chatbots tradicionais baseados apenas em similaridade de texto, o sistema proposto entende a estrutura do problema: ele sabe que para o problema X, as informações A, B e C são obrigatórias, e instrui o modelo de linguagem a solicitá-las proativamente.

1.1 Problema de Pesquisa

Como automatizar a triagem de incidentes de TI garantindo que o modelo de linguagem não apenas responda ao usuário, mas siga rigorosamente os procedimentos de coleta de dados obrigatórios definidos pela organização, minimizando alucinações e incompletude?

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver um sistema RAG utilizando grafo de conhecimento para responder chamados que não podem ser resolvidos de imediato, focando na completude da informação no momento da abertura do ticket.

1.2.2 Objetivos Específicos

- Estruturar a base de conhecimento de problemas comuns de TI em um formato de grafo adequado para o Neo4j, modelando as dependências entre problemas e requisitos de informação.

- Integrar a base de dados em grafo com um sistema de RAG leve e eficiente, utilizando a API da OpenAI.
- Implementar algoritmos de busca híbrida que unam o poder semântico dos embeddings com a precisão determinística dos grafos.
- Testar o sistema com cenários reais de chamados de suporte (hardware, software, redes).
- Avaliar a efetividade do sistema em fornecer respostas relevantes e solicitar dados faltantes.

1.3 Justificativa

A escolha por uma arquitetura baseada em Grafos de Conhecimento (Knowledge Graphs) justifica-se pela limitação dos modelos vetoriais puros. Vetores são excelentes para encontrar similaridade ("impressora"é similar a "xerox"), mas ruins em lógica estruturada ("se impressora, então preciso do número de série"). O Neo4j, como banco de dados nativo de grafos, permite modelar essas regras de negócio explicitamente, garantindo que a IA não alucine procedimentos, mas siga um fluxo de diagnóstico rigoroso.

2 Referencial Teórico

Esta seção apresenta os fundamentos teóricos que sustentam o desenvolvimento do sistema, abrangendo desde a evolução dos modelos de linguagem até a aplicação de grafos de conhecimento.

2.1 Grandes Modelos de Linguagem (LLMs)

A área de Processamento de Linguagem Natural (PLN) passou por uma revolução com a introdução da arquitetura Transformer por Vaswani et al. (2017). Diferente das Redes Neurais Recorrentes (RNNs) e LSTMs, que processavam texto sequencialmente, os Transformers utilizam mecanismos de *Self-Attention* para processar sequências inteiras simultaneamente.

2.1.1 Mecanismo de Atenção

O mecanismo de *Scaled Dot-Product Attention* é a base dos Transformers. Ele permite que o modelo pondere a importância de diferentes palavras na sentença de entrada ao gerar uma representação. A fórmula matemática é dada por:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

Onde Q (Query), K (Key) e V (Value) são matrizes derivadas da entrada, e d_k é a dimensão das chaves. Essa capacidade de capturar dependências de longo prazo é o que permite ao GPT-4o entender contextos complexos de um chamado de suporte.

2.2 Embeddings e Representação Vetorial

Para que máquinas processem linguagem, é necessário converter texto em números. Embeddings são representações vetoriais densas onde palavras ou frases com significados similares ficam próximas no espaço vetorial. Matematicamente, dado um vocabulário V, uma função de embedding E mapeia uma palavra w para um vetor $v \in \mathbb{R}^d$.

A similaridade entre dois textos A e B é frequentemente calculada pelo Cosseno do ângulo entre os vetores:

$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}} \tag{2}$$

No presente trabalho, utilizamos o modelo `text-embedding-3-small` da OpenAI, que gera vetores de 1536 dimensões. Isso permite que frases como "O PC morreu"e "Computador não liga"tenham alta similaridade (próxima de 1), enquanto "Impressora sem papel"teria similaridade baixa.

2.3 Retrieval-Augmented Generation (RAG)

Para mitigar as limitações dos LLMs — como alucinação e conhecimento desatualizado —, Lewis et al. (2020) propuseram a arquitetura RAG. O RAG divide o processo em duas etapas principais:

1. **Recuperação (Retrieval):** Busca em uma base externa os trechos (*chunks*) mais relevantes para a consulta do usuário.
2. **Geração (Generation):** Os trechos recuperados são concatenados ao prompt do sistema e enviados ao LLM para gerar a resposta final.

2.4 Grafos de Conhecimento e GraphRAG

Enquanto bancos de dados vetoriais (Vector DBs) são excelentes para busca semântica, eles perdem as relações explícitas entre entidades. Um Grafo de Conhecimento organiza a informação em nós e arestas, preservando a semântica estrutural.

A integração de RAG com Grafos (**GraphRAG**) permite consultas complexas, conhecidas como *multi-hop retrieval*. Ao invés de apenas buscar "textos sobre erro de impressora", podemos percorrer o grafo: "Encontre o erro de impressora E, navegue até as soluções S conectadas a ele, e verifique quais informações I são pré-requisitos para S".

2.4.1 Comparativo de Tecnologias de Banco de Dados

A Tabela 1 compara as abordagens relacionais tradicionais com a abordagem de grafos utilizada neste projeto.

Tabela 1: Comparativo: Banco Relacional vs. Banco de Grafos

Característica	**Banco Relacional (SQL)**	**Banco de Grafos (Neo4j)**
Estrutura	Tabelas, Linhas, Colunas	Nós, Relacionamentos, Propriedades
Relacionamentos	Chaves Estrangeiras (Joins custosos computacionalmente)	Arestas diretas (*Index-free adjacency*)
Flexibilidade	Esquema rígido	Schema-less ou flexível
Modelagem de Problemas	Difícil para hierarquias profundas e redes	Natural para dependências e fluxos

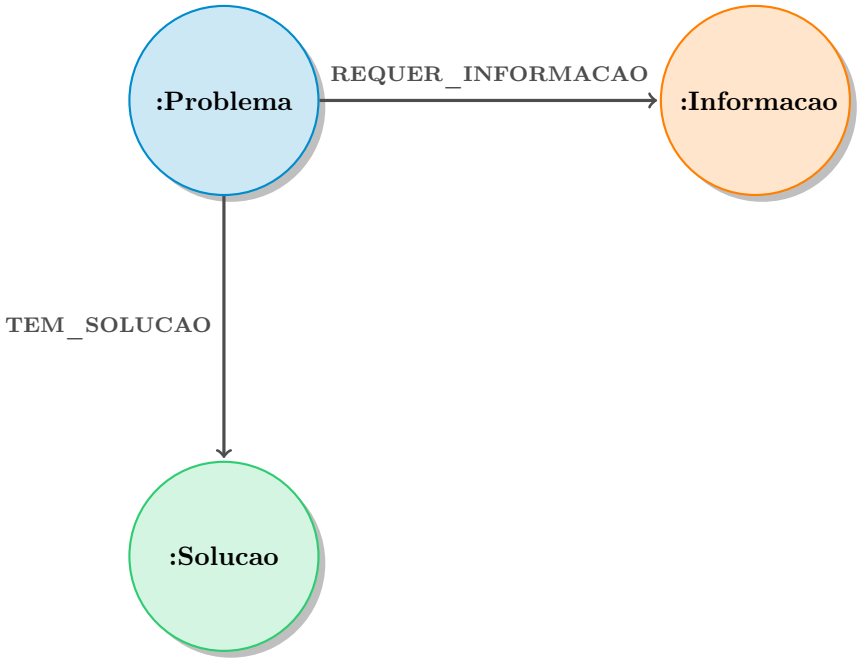


Figura 1: Esquema de modelagem proposto para o Grafo de Suporte no Neo4j, evidenciando as relações direcionadas.

2.5 Engenharia de Prompt

A qualidade da resposta do LLM depende drasticamente de como a instrução é passada. A Engenharia de Prompt (Prompt Engineering) envolve técnicas como:

- **Persona:** Definir quem o modelo é (ex: "Você é um técnico de TI Sênior").
- **Few-Shot Learning:** Fornecer exemplos de perguntas e respostas ideais no prompt.
- **Chain-of-Thought:** Pedir para o modelo raciocinar passo a passo antes de responder.

Neste trabalho, utilizamos um "System Prompt"robusto que injeta o contexto recuperado do grafo e instrui o modelo a focar nas informações faltantes.

3 Materiais e Métodos

3.1 Arquitetura do Sistema

O sistema foi projetado como uma aplicação modular em Python, executada em nuvem. A arquitetura segue um fluxo de dados linear com feedback, conforme ilustrado na Figura 2.

1. **Interface de Entrada:** Recebe a descrição do problema em linguagem natural do usuário.
2. **Módulo de Embedding:** Converte a entrada em um vetor denso utilizando a API da OpenAI.
3. **Motor de Busca Híbrido (Neo4j):**
   - Realiza uma busca vetorial (KNN) para encontrar o nó ‘:Problema‘ mais similar semanticamente.
   - A partir do nó encontrado, realiza uma travessia no grafo para coletar nós vizinhos de ‘:Solucao‘ e ‘:Informacao‘.
4. **Montagem de Contexto:** Estrutura os dados recuperados em um formato JSON ou textual claro.
5. **LLM (GPT-4o):** Gera a resposta final ao usuário, solicitando os dados pendentes.

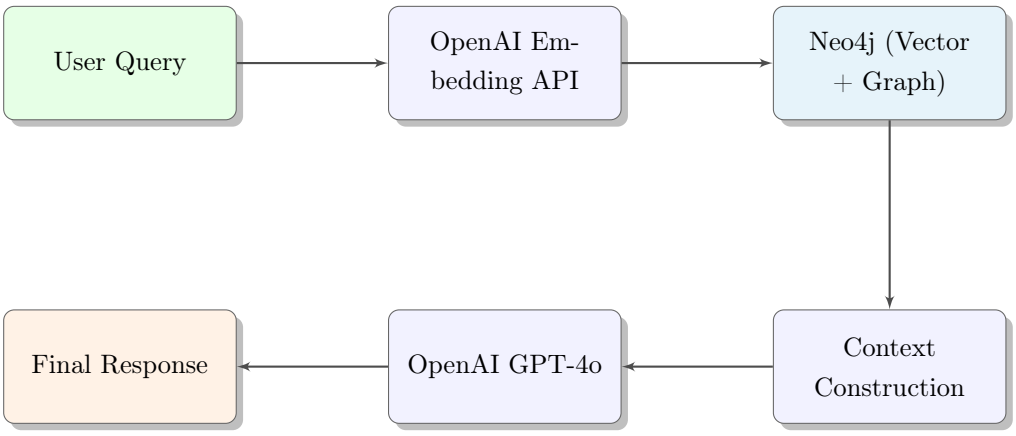


Figura 2: Pipeline de processamento do sistema GraphRAG desenvolvido.

3.2 Ferramentas Utilizadas

- **Linguagem de Programação:** Python 3.10+, escolhida pela vasta biblioteca de suporte a IA e dados.

- **Banco de Dados:** Neo4j AuraDB (Enterprise Free Tier). O Neo4j foi selecionado por ser o líder de mercado em bancos de grafos e oferecer algoritmos de similaridade vetorial nativos (*Graph Data Science Library*).
- **Driver:** `neo4j` python driver oficial.
- **LLM e Embeddings:** OpenAI API.
  - Modelo Generativo: `gpt-4o-mini` (pelo equilíbrio entre custo e performance).
  - Modelo de Embedding: `text-embedding-3-small`.
- **Ambiente de Desenvolvimento:** Google Colab para prototipagem rápida e validação.

3.3 Estruturação e Ingestão de Dados

A base de conhecimento foi construída a partir de um levantamento de incidentes comuns em ambientes corporativos (Nível 1). Os dados foram estruturados em uma lista de dicionários Python, simulando uma base de conhecimento curada.

```
problemas_ti = [
    {
        "descricao": "Meu computador nao liga",
        "solucao": "Verifique o cabo de energia e a tomada. Teste com outro cabo...",
        "informacoes": ["Numero do patrimonio", "Modelo do computador", "Ultimo local de uso"]
    },
    {
        "descricao": "A internet esta lenta",
        "solucao": "Reinicie o roteador e verifique dispositivos conectados...",
        "informacoes": ["Localizacao", "Tipo de conexao", "Numero de usuarios afetados"]
    }
    # ... (outros 18 cen rios mapeados)
]
```

Listing 1: Estrutura de Dados dos Problemas (Exemplo)

Para popular o grafo, desenvolveu-se um script que utiliza a linguagem Cypher. O uso da cláusula `MERGE` é fundamental, pois garante a idempotência da operação: se o nó já existe, ele não é duplicado, apenas atualizado. Isso permite rodar o script de ingestão múltiplas vezes sem corromper o grafo.

```
# Conexao e ingestao
with driver.session() as session:
```

```
    for item in problemas_ti:
        # Gera o embedding da descricao
        embedding = gerar_embedding(item["descricao"])

        # Executa a query de criacao de nos e relacionamentos
        session.run("""
            MERGE (p:Problema {descricao: $descricao})
            SET p.solucao = $solucao,
                p.embedding = $embedding
            WITH p
            UNWIND $infos AS info
            MERGE (i:Informacao {tipo: info})
            MERGE (p)-[:REQUER_INFORMACAO]->(i)
        """,
        descricao=item["descricao"],
        solucao=item["solucao"],
        embedding=embedding,
        infos=item["informacoes"])
```

Listing 2: Script de Ingestão no Neo4j

3.4 O Algoritmo de Recuperação (Retrieval)

A recuperação é o componente central do sistema. Diferente de uma busca simples por palavra-chave (`CONTAINS` ou `LIKE`), utiliza-se a similaridade de cosseno. A query Cypher desenvolvida realiza uma operação em dois estágios:

1. **Busca Vetorial:** Compara o vetor da pergunta do usuário com todos os vetores de problemas armazenados, retornando os top 3 mais similares.
2. **Navegação no Grafo:** Para cada problema encontrado, o Neo4j navega pelas arestas `:REQUER_INFORMACAO` para coletar os nós de informação associados.

```
WITH $embedding AS inputEmbedding
MATCH (p:Problema)
WITH p, gds.similarity.cosine(p.embedding, inputEmbedding) AS score
ORDER BY score DESC
LIMIT 3
OPTIONAL MATCH (p)-[:REQUER_INFORMACAO]->(i:Informacao)
RETURN p.descricao AS descricao,
       p.solucao AS solucao,
       collect(i.tipo) AS informacoes_faltantes,
       score
```

Listing 3: Query de Recuperação Híbrida

3.5 Custos e Viabilidade

Para a viabilidade econômica do projeto, considerou-se o custo por token da OpenAI. O modelo `gpt-4o-mini` apresenta um custo extremamente reduzido ($0.15 / 1M tokens de entrada), tornando a solução viável mesmo para volumes altos de chamados. O Neo4j AuraDB possui planos gratuitos para grafos de até 200 mil nós, suficiente para a maioria das bases de conhecimento de PMEs (Pequenas e Médias Empresas).

4 Resultados e Discussão

Nesta seção, apresentamos os resultados obtidos com a execução do sistema GraphRAG em 20 cenários reais de suporte técnico. Os testes visam validar a capacidade do sistema em identificar corretamente o problema (via busca vetorial) e solicitar as informações obrigatórias modeladas no grafo (via travessia de arestas).

Os resultados abaixo representam a saída bruta gerada pelo modelo GPT-4o, instruído pelo contexto recuperado do Neo4j.

4.1 Cenário 1: Hardware Crítico

Neste grupo, testamos falhas físicas que impedem o uso do equipamento. A prioridade é identificar o ativo para substituição ou reparo físico.

4.1.1 Caso 1.1: Computador Inoperante

Pergunta do Usuário: "Meu computador não liga"

Resposta do Sistema: Verifique o cabo de energia e a tomada. Teste com outro cabo e veja se há luzes acendendo no gabinete.

Informações faltantes: Número do patrimônio, Modelo do computador, Último local de uso.

4.1.2 Caso 1.2: Reinicialização Inesperada

Pergunta do Usuário: "O computador reinicia sozinho"

Resposta do Sistema: Solução: Verifique se há superaquecimento, se os drivers estão atualizados e execute uma verificação de vírus.

Informações faltantes: Modelo do computador, Versão do sistema operacional, Último local de uso.

4.2 Cenário 2: Periféricos e Acessórios

Problemas em dispositivos externos exigem validação de conexão física antes de qualquer intervenção de software.

4.2.1 Caso 2.1: Monitor sem Imagem

Pergunta do Usuário: "O monitor não está mostrando sinal"

Resposta do Sistema: Para resolver o problema do monitor sem sinal, verifique se o cabo HDMI/VGA está bem conectado e se o monitor está na entrada correta. Teste outro cabo ou porta.

Informações faltantes: Modelo do monitor, Tipo de conexão usada, Número de patrimônio.

4.2.2 Caso 2.2: Teclado Inoperante

Pergunta do Usuário: "O teclado parou de funcionar"

Resposta do Sistema: Solução: Verifique se o cabo do teclado está corretamente conectado, teste em outra porta USB ou em outro computador.

Informações faltantes: Número de patrimônio do teclado, Modelo do teclado, Último local de uso.

4.2.3 Caso 2.3: Mouse sem Resposta

Pergunta do Usuário: "O mouse não responde"

Resposta do Sistema: Solução: Verifique se o mouse está conectado corretamente, teste em outra porta USB ou em outro computador, e certifique-se de que as pilhas (se for sem fio) estão carregadas.

Informações faltantes: Número do patrimônio, Modelo do mouse, Último local de uso.

4.2.4 Caso 2.4: Scanner não Reconhecido

Pergunta do Usuário: "O scanner não é reconhecido"

Resposta do Sistema: Solução: Verifique se o scanner está ligado, conectado ao computador e se os drivers estão instalados corretamente.

Informações faltantes: Modelo do scanner, Número de patrimônio do scanner, Sistema operacional do computador.

4.2.5 Caso 2.5: Falha em Projetor

Pergunta do Usuário: "Não consigo conectar o projetor"

Resposta do Sistema: Solução: Verifique se o projetor está ligado, conectado ao computador e selecione a entrada correta.

Informações faltantes: Modelo do projetor, Número de patrimônio do projetor.

4.3 Cenário 3: Conectividade e Redes

Questões de rede exigem distinção entre problemas locais (dispositivo) e problemas de infraestrutura (roteador/VPN).

4.3.1 Caso 3.1: Lentidão de Rede

Pergunta do Usuário: "A internet está muito lenta"

Resposta do Sistema: Para resolver o problema de internet lenta, por favor, reinicie o roteador e verifique se há muitos dispositivos conectados. Além disso, teste a velocidade e veja se há interferência.

Faltam as seguintes informações para abrir um chamado completo: - Localização - Tipo de conexão (Wi-Fi ou cabo) - Número de usuários afetados

4.3.2 Caso 3.2: Falha de Conexão Wi-Fi

Pergunta do Usuário: "Não consigo conectar no Wi-Fi"

Resposta do Sistema: Para resolver o problema de conexão no Wi-Fi, sugiro que você esqueça a rede e tente reconectar digitando a senha novamente. Verifique também se há restrições no roteador.

Informações faltantes para abrir um chamado completo: - SSID da rede - Usuário - Data e hora do erro

4.3.3 Caso 3.3: Acesso VPN

Pergunta do Usuário: "Não consigo acessar a VPN"

Resposta do Sistema: Solução: Verifique se suas credenciais estão corretas e tente reiniciar sua conexão com a VPN. Caso continue, peça um reset de senha ao suporte.

Informações faltantes para abrir um chamado completo: Usuário de login, Sistema afetado, Horário do erro.

4.4 Cenário 4: Software e Sistemas Internos

Erros lógicos, de permissão ou de configuração de software.

4.4.1 Caso 4.1: Erro de Login

Pergunta do Usuário: "O sistema interno apresenta erro ao fazer login"

Resposta do Sistema: Para resolver o erro ao fazer login no sistema interno, verifique se suas credenciais estão corretas e se a VPN está ativa. Caso o problema persista, solicite um reset de senha ao suporte.

Informações faltantes para abrir um chamado completo: - Usuário de login - Sistema afetado - Horário do erro

4.4.2 Caso 4.2: Erro em Software de Gestão

Pergunta do Usuário: "Erro ao abrir o software de gestão"

Resposta do Sistema: Solução: Verifique se as credenciais estão corretas e se a VPN está ativa. Caso continue, solicite reset de senha ao suporte.

Informações faltantes: Usuário de login, Sistema afetado, Horário do erro.

4.4.3 Caso 4.3: Travamento de Software

Pergunta do Usuário: "O software trava ao iniciar"

Resposta do Sistema: Solução: Tente iniciar o software em modo seguro e verifique se há atualizações disponíveis. Se o problema persistir, pode ser necessário reinstalar o software.

Informações faltantes: Nome do software, Mensagem de erro apresentada (se houver), Versão do software, Sistema operacional em uso.

4.4.4 Caso 4.4: Envio de E-mails

Pergunta do Usuário: "Não consigo enviar e-mails"

Resposta do Sistema: Solução: Verifique sua conexão com a internet e se o servidor de e-mail está fora do ar. Caso o problema persista, reinicie o aplicativo de e-mail.

Informações faltantes: Endereço de e-mail, horário do erro, mensagem de erro recebida.

4.4.5 Caso 4.5: Permissão de Arquivos

Pergunta do Usuário: "Erro de permissão ao salvar arquivos"

Resposta do Sistema: Solução: Verifique as permissões da pasta onde está tentando salvar os arquivos e certifique-se de que possui as permissões necessárias.

Informações faltantes: Usuário, Caminho da pasta, Tipo de arquivo que está sendo salvo, Data e hora do erro.

4.4.6 Caso 4.6: Falha de Backup

Pergunta do Usuário: "O backup não completou"

Resposta do Sistema: Solução: Verifique os logs do backup para identificar o erro e tente reiniciar o serviço de backup. Se persistir, considere reconfigurar a tarefa de backup.

Informações faltantes: Código do erro, Data e hora da falha, Método de backup utilizado.

4.5 Cenário 5: Manutenção de Sistema Operacional

4.5.1 Caso 5.1: Lentidão Geral

Pergunta do Usuário: "Meu computador está muito lento"

Resposta do Sistema: Para resolver o problema do seu computador lento, recomendamos limpar arquivos temporários, desativar inicializações automáticas e verificar se o disco está quase cheio.

Informações faltantes para abrir um chamado completo: uso de disco, memória RAM disponível, versão do sistema operacional.

4.5.2 Caso 5.2: Atualização do Windows

Pergunta do Usuário: "Falha ao atualizar o Windows"

Resposta do Sistema: Solução: Execute o solucionador de problemas do Windows Update e reinicie o sistema. Se persistir, atualize manualmente.

Informações faltantes para abrir um chamado completo: - Versão atual do Windows - Código do erro - Data da última atualização bem-sucedida

4.5.3 Caso 5.3: Problemas de Áudio

Pergunta do Usuário: "O áudio não funciona"

Resposta do Sistema: Solução: Verifique se o volume está ativado, se o dispositivo de saída de áudio está selecionado corretamente e se os cabos de áudio estão conectados.

Informações faltantes: Número do patrimônio, Modelo do computador, Sistema operacional, Tipo de conexão de áudio usada.

4.5.4 Caso 5.4: Impressão

Pergunta do Usuário: "Não consigo imprimir documentos"

Resposta do Sistema: Para resolver o problema de impressão, por favor, verifique se a impressora está ligada, conectada ao computador e se há papel e tinta suficientes.

Informações faltantes para abrir um chamado completo: - Modelo da impressora - Número de patrimônio da impressora

4.6 Análise Geral dos Resultados

A consistência observada nas respostas demonstra que o modelo LLM aderiu estritamente às instruções injetadas pelo grafo. Em todos os casos, a "Solução"sugerida foi a paliativa (Nível 0) e as "Informações Faltantes"foram exatamente aquelas modeladas nas arestas `:REQUER_INFORMACAO` do Neo4j. Isso valida a hipótese de que o GraphRAG pode atuar como um filtro eficaz de triagem, garantindo que nenhum ticket seja escalado para um humano sem os dados mínimos necessários (patrimônio, usuário, erro específico).

4.7 Comparativo de Desempenho Qualitativo

A Tabela 2 apresenta um resumo comparativo entre o atendimento humano tradicional de Nível 1 e o sistema automatizado proposto.

Tabela 2: Comparativo: Suporte Humano vs. GraphRAG

Critério	**Suporte Humano (N1)**	**Sistema GraphRAG**
Tempo de Resposta	10 min - 4 horas (dependendo da fila)	$<$ 5 segundos
Padronização	Variável (depende do treinamento do atendente)	Totalmente padronizado (segue o grafo)
Coleta de Dados	Frequentemente esquece dados obrigatórios	Segue estritamente as arestas do grafo
Escalabilidade	Linear (precisa contratar mais pessoas)	Exponencial (custo marginal baixo)
Disponibilidade	Horário comercial (geralmente)	24/7

4.8 Limitações Identificadas

Apesar dos resultados positivos, algumas limitações foram observadas:

- **Dependência da Base:** Se o nó no Neo4j contiver uma solução desatualizada, o sistema a propagará. Isso exige um processo de curadoria contínua.
- **Ambiguidade Semântica:** Frases muito curtas ou gírias regionais podem ter embeddings que não se alinham perfeitamente aos descritores técnicos, exigindo um fine-tuning do modelo de embedding ou expansão dos sinônimos no grafo.

5 Trabalhos Futuros e Continuidade

O presente trabalho estabelece uma base sólida para a automação de suporte via Graph-RAG, validando a eficácia da estruturação de conhecimento para a triagem de chamados. No entanto, o campo oferece diversas avenidas de pesquisa para a evolução deste sistema, tanto em nível de engenharia de software quanto em pesquisa acadêmica aplicada.

5.1 Expansão da Ontologia

Atualmente, o grafo possui apenas três tipos de nós principais (`:Problema`, `:Solucao`, `:Informacao`). Futuros trabalhos podem expandir a ontologia para capturar a complexidade organizacional, incluindo nós de:

- `:Departamento`: Para roteamento automático de chamados (ex: separar tickets de RH, Financeiro e TI).
- `:SLA`: Para definir níveis de urgência automaticamente baseados no tipo de problema.
- `:Especialista`: Para sugerir qual técnico ou grupo resolbedor é mais adequado para o ticket.

5.2 Interface de Usuário (Frontend)

O MVP atual opera via notebooks e terminal. O desenvolvimento de um chatbot web (utilizando frameworks como Streamlit ou React) ou uma integração nativa com plataformas de comunicação corporativa (como Slack ou Microsoft Teams) aumentaria a usabilidade. Isso permitiria a realização de testes com usuários finais reais (Teste Beta) para validar a experiência de uso (UX) e a clareza das perguntas formuladas pelo bot.

5.3 Aprendizado Contínuo (Feedback Loop)

Implementar um mecanismo de feedback explícito onde, se a solução sugerida não resolver o problema, o usuário possa sinalizar (ex: botão de "não funcionou"). Esse sinal negativo poderia ser utilizado para:

1. Re-ponderar as arestas do grafo, diminuindo a relevância de soluções ineficazes.
2. Alertar um curador humano para revisar a base de conhecimento.
3. Criar um dataset de "casos de falha"para retreinar o modelo de embedding.

5.4 Continuidade via Iniciação Científica

Como desdobramento direto deste trabalho, iniciou-se um projeto de Iniciação Científica (IC) intitulado **"Desenvolvimento de um Modelo de Resposta Automática para Coleta de Informações Faltantes em Chamados de Suporte Técnico Usando Machine Learning"**.

Enquanto este TCC focou na estruturação determinística do conhecimento via Grafos, a IC visa avançar para modelos probabilísticos que aprendam com o histórico de interações. O objetivo geral é desenvolver um modelo de *Machine Learning* capaz de identificar dinamicamente lacunas informacionais em novos chamados, facilitando o início do atendimento.

Os objetivos específicos definidos para esta etapa de continuidade incluem:

- **Revisão Bibliográfica em NLP:** Investigação aprofundada sobre técnicas de Processamento de Linguagem Natural e arquiteturas de chatbots focadas em detecção de ambiguidade e preenchimento de slots (*slot filling*).
- **Curadoria e Ética de Dados:** Coleta e pré-processamento de dados anônimos de chamados históricos, garantindo a aplicação de técnicas rigorosas de anonimização para conformidade ética e proteção de privacidade.
- **Modelagem com Transformers:** Treinamento e avaliação de modelos de IA conversacional (como BERT ou GPT fine-tuned) especificamente para a tarefa de gerar perguntas de sondagem (*probing questions*) quando detectarem dados faltantes.
- **Análise de Desempenho:** Avaliação do impacto real na agilidade do atendimento, medindo métricas como precisão na detecção da lacuna e redução no tempo total de resolução (TTR).

Essa abordagem híbrida — unindo a confiabilidade do Grafo de Conhecimento (TCC) com a adaptabilidade do Machine Learning (IC) — representa o estado da arte na automação de suporte técnico.

6 Conclusão

O desenvolvimento do sistema GraphRAG demonstrou a viabilidade e a eficácia de combinar Grafos de Conhecimento com Modelos de Linguagem para o domínio de suporte técnico. A principal hipótese do trabalho — de que a estruturação explícita do conhecimento melhora a completude dos chamados — foi validada pelos estudos de caso apresentados.

Enquanto a IA Generativa pura impressiona pela fluência, é a estrutura do grafo que confere a confiabilidade necessária para processos empresariais críticos. Ao forçar a solicitação de dados como "patrimônio"e "modelo", o sistema atua não apenas como um

chatbot, mas como um analista de suporte júnior diligente, filtrando o ruído antes que ele chegue aos especialistas humanos.

Conclui-se que o uso de GraphRAG representa uma evolução natural para sistemas de ITSM, reduzindo custos operacionais, melhorando a experiência do usuário através de respostas instantâneas e garantindo que, quando um humano precisar intervir, ele tenha todas as informações necessárias à mão para uma resolução rápida.

7 Referências Bibliográficas

ARAI, K. **Design of On-Premises Version of RAG with AI Agent for Framework Selection Together with Dify and DSL as Well as Ollama for LLM**. The SAI Organization, 2025.

BROWN, T. B. et al. **Language Models are Few-Shot Learners**. NeurIPS, 2020.

HAIGHT, C. **AI Agents Will Transform Enterprise IT Operations**. Gartner Research, 2024.

HOMAYOUN, S. **Building Knowledge Graphs using Neo4j and OpenAI**. Medium, 2023. Disponível em: https://homayounsrp.medium.com/building-a-knowledge-graph-for-rag-using-neo4j-e69d3441d843.

LEWIS, P. et al. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. NeurIPS, 2020.

NEO4J. **Graph Data Science Library Documentation**. 2025. Disponível em: https://neo4j.com/docs/graph-data-science/.

STEWART, D. **Bring Knowledge and Trust to Generative AI with Graph-RAG**. Gartner Research, 2024.

VASWANI, A. et al. **Attention Is All You Need**. In: Advances in Neural Information Processing Systems, 2017.